

Performance Testing

Performance Testing

Date	14 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how efficiently and reliably the Personalized Networking Assistant responds during important application workflows. The focus is on the implemented React and TypeScript frontend, Node.js and Express backend, SQLite persistence, Google Gemini generation, and Wikipedia factual verification. This document explains the testing approach without claiming unverified benchmark values.

Step 1: Testing Overview

Field	Details
Testing Type	Performance, responsiveness, stability, and concurrent-request evaluation
Application Under Test	Personalized Networking Assistant full-stack web application
Backend	Node.js and Express API
External Services	Google Gemini for AI generation and Wikipedia for factual context
Persistence	SQLite database
Deployment Context	Application workflow designed for local development and deployed operation on Render

Step 2: Core Performance Scenarios

S.No	Scenario	What Is Evaluated	Expected Behaviour
1	Authentication and Profile	Registration, login, profile retrieval, and profile updates.	Requests complete correctly without unnecessary delay or data inconsistency.
2	Gemini Content Generation	Server-side generation using user profile and event context.	The interface remains responsive while generation is processed and results are returned correctly.
3	Wikipedia Verification	External factual lookup for a user-provided networking topic.	The application handles external-service latency and returns factual context or a controlled error.
4	History and Feedback	SQLite reads and writes for generated sessions, ratings, likes, dislikes, and comments.	Data is stored and retrieved consistently during repeated use.

Step 3: Performance Test Results

Step 3: Performance Evaluation Criteria

Metric / Area	Purpose	How It Should Be Interpreted
Response Time	Measures the time between a user action and the corresponding application response.	Local API operations are expected to be faster than AI generation or external factual lookup, which depend on third-party services and network conditions.
Throughput	Measures how many requests the backend can process over a period of time.	Useful when testing repeated or concurrent API requests. Results should be recorded from an actual load-testing run rather than estimated.
Error Rate	Tracks failed requests, invalid responses, and server errors.	A low error rate indicates stability, but failures from external providers must be distinguished from internal application failures.
Database Performance	Evaluates SQLite read/write behaviour for profiles, history, and feedback.	Repeated operations should remain consistent. Higher concurrency may require future migration to a managed relational database.
External API Latency	Separates Gemini and Wikipedia response time from internal Express processing.	This helps identify whether delays originate in the application or in an external service.

Important Result-Reporting Rule

The uploaded sample contained specific benchmark numbers and technologies that are not reliable evidence for the current implementation. Therefore, those figures are not repeated here. Actual average response time, maximum response time, throughput, CPU usage, memory usage, and error rate should be entered only after a real test run against the current Node.js/Express application and its deployed environment.

Step 4: Test Execution Method

1. Prepare the Environment

Run the current application with the required environment variables configured. Confirm that the Express backend, SQLite database, Gemini integration, and Wikipedia lookup are functioning before performance measurement.

2. Select Representative Endpoints

Test a mix of internal operations and external-service operations. Internal examples include authentication, profile access, history retrieval, and feedback storage. External examples include Gemini generation and Wikipedia verification.

3. Establish a Baseline

Send individual requests first and record normal behaviour. This baseline helps distinguish ordinary external API latency from degradation caused by concurrent load.

4. Increase Request Load Gradually

Run repeated and concurrent requests in controlled stages. Observe response time, successful requests, failed requests, and whether the application remains stable.

5. Separate External Dependencies

Record Gemini and Wikipedia operations separately from local database and API operations because their response times depend on network conditions and third-party availability.

6. Review Persistence Behaviour

Verify that concurrent or repeated history and feedback operations do not create missing, duplicated, or inconsistent records in SQLite.

7. Record Actual Evidence

Save the load-test report, server logs, timestamps, and measured metrics. Only measured values should be placed in the final results table.

Step 5: Expected Bottlenecks and Analysis

Area	Why It Can Affect Performance	Recommended Observation / Improvement
Google Gemini	AI generation is an external network request and may vary with provider latency, model processing time, quotas, or temporary service conditions.	Measure separately from internal API time; handle timeouts and errors clearly; avoid unnecessary duplicate generation requests.
Wikipedia	Factual verification depends on network availability and the external Wikipedia service.	Use controlled error handling and consider safe caching for repeated factual lookups where appropriate.
SQLite	SQLite is effective for the current application but can become a constraint under high concurrent write activity.	Monitor database operations and consider PostgreSQL or another managed relational database if concurrency requirements grow.
Server Resources	CPU and memory usage can rise with concurrent requests, serialization, logging, and application workload.	Measure resource usage in the actual deployment environment and scale infrastructure based on observed demand.
Frontend Perception	Even when backend work is valid, long AI operations can feel slow without clear feedback.	Keep loading states and progress feedback visible so users understand that generation or verification is in progress.

Analysis Principle

Performance should be interpreted by workflow. Authentication, profile, history, and feedback operations primarily measure the Express and SQLite layers, while Gemini generation and Wikipedia verification also include unavoidable external-service latency. Combining all of these into one response-time figure can give a misleading picture of the system.

Step 6: Performance Test Result Template

The following table is intentionally left without invented benchmark values. Enter the measurements obtained from the actual performance-testing run.

Metric	Actual Measured Value	Status	Remarks
Average Response Time	To be measured	Pending actual test	Record separately for internal APIs and external-service workflows.
Maximum Response Time	To be measured	Pending actual test	Identify the endpoint and conditions that produced the maximum value.
Throughput	To be measured	Pending actual test	Record requests per second from the selected load-testing configuration.
Error Rate	To be measured	Pending actual test	Separate application failures from Gemini, Wikipedia, or network failures.
CPU Utilization	To be measured	Pending actual test	Capture from the environment where the backend is actually running.
Memory Utilization	To be measured	Pending actual test	Observe memory behaviour during sustained and concurrent requests.

Conclusion

A valid performance assessment of the Personalized Networking Assistant should measure the current Node.js/Express implementation directly, distinguish internal processing from Gemini and Wikipedia latency, verify SQLite behaviour under repeated use, and report only observed results. This approach provides a reliable basis for future optimization and scaling decisions.